MoodTrip Project Report

Mood and Budget Aware AI Travel Itinerary Planner

Provisional project report | 15 October 2026

Submitted by [Student names and registration numbers]

Course [Course name and code]

Department [Department name]

Institution [Institution name]

Guide [Guide name]

Live application
https://mood-trip-mu.vercel.app/

Report contents

Abstract and project overview 2

Requirements and system design 3

AI workflow and recommendation method 4

Implementation and data management 5

Testing and deployment 6

Limitations conclusion and references 7

Abstract and project overview

MoodTrip is a web application that creates a day-by-day travel outline using a destination, trip duration, budget, and travel mood. It combines place data from Geoapify with AI-assisted sequencing through OpenRouter. The backend validates generated plans and stores them in Supabase. The deployed website is available at https://mood-trip-mu.vercel.app/.

The project addresses the effort required to combine place discovery, personal preferences, and an approximate activity budget into one plan. A traveler selects one of five moods: relaxing, adventurous, cultural, foodie, or nightlife. The application returns three suggested stops per day, with visit times, duration estimates, activity or meal costs, and Google Maps links.

Problem statement

A list of attractions alone does not tell a traveler how to arrange visits around a preferred pace and spending limit. MoodTrip explores how a structured form, mood-specific place categories, and a language model can turn these inputs into a practical first draft for a trip.

Objectives

- Collect destination, duration, budget, and mood through a guided planning interface.
- Ground suggested stops in retrieved place records rather than unrestricted venue generation.
- Validate the AI response before displaying and saving an itinerary.
- Support star ratings, read-only share links, and browser-based PDF export.

Scope

The prototype supports 1 to 7 days and a whole-trip budget of INR 1,000 to INR 50,000 per person for the listed activities and meals. It does not book travel or process payments. Flights, hotels, transport, and unlisted meals are outside the budget calculation. Costs and visit times are suggestions that users must verify.

Requirements and system design

Functional requirements

- The user can enter a destination, choose trip length and budget, and select a mood.
- The application resolves the destination, retrieves candidate places, and requests a structured itinerary.
- The user can view separate days, follow map links, confirm a plan, and print all days to PDF.
- The creator can save or update a rating and create a share link using the private feedback code associated with the plan.

Quality requirements

The interface should remain readable on phones and computers, show loading and error states, and keep service credentials on the server. Invalid input and invalid AI output should be rejected. External services may be slow or unavailable, so the interface needs understandable failure messages. These are design requirements; no formal availability or performance target has been measured.

Architecture

The browser runs a React single-page application. Requests to /api are handled by an Express application hosted as a Vercel function. The backend contacts Geoapify for geocoding and places, OpenRouter for generation, and Supabase for stored plans and ratings. Responses return to the browser through the same website domain.

1 React interface
2 Express API on Vercel
3 Geoapify places and mood ranking
4 OpenRouter generation and server validation
5 Supabase storage and browser result

Technology choices

React and React Router implement the pages and navigation. Vite builds the frontend; Tailwind CSS, Framer Motion, and Lucide React support styling, animation, and icons. Node.js and Express implement the API. Supabase provides PostgreSQL storage. GitHub holds the source, and Vercel deploys the frontend and backend. The current AI route is openrouter/free; Gemini was explored during development but is not the active generation path.

AI workflow and recommendation method

Generation pipeline

- Validate a non-empty destination of at most 80 characters, a duration of 1 to 7 days, a permitted budget, and a supported mood.
- Resolve the destination with Geoapify and search within 15 km of its center for mood-related places and food options.
- Assign initial mood scores from place categories, then adjust candidate rankings using available same-mood itinerary feedback.
- Send a shortlist of candidate places, trip constraints, and a JSON schema to OpenRouter. Request three stops per day in morning, afternoon, and evening periods.
- Validate the response, attach trusted place metadata, save the plan, and return it with a private creator feedback code.

Feedback ranking

The recommendation method is a rule-based and statistical ranker, not a separately trained machine-learning model. It reads up to 200 recent rated itineraries for the selected mood and counts each place once per rated itinerary. An itinerary rating is an indirect signal about every place in that itinerary.

Smoothed rating = (sum of ratings + 9) / (rating count + 3). Feedback adjustment = (smoothed rating - 3) x 5. Final ranking score = initial mood score + feedback adjustment.

The three neutral prior ratings reduce the influence of a single review. For example, a place with one five-star rating has a smoothed rating of 3.5 and a feedback adjustment of 2.5. The language model uses the ranked shortlist but is not guaranteed to select places in ranking order. If feedback retrieval fails, generation can continue using the initial scores.

Output validation

The backend checks the day count, required structure, candidate place IDs, repeated IDs, periods, times, visit durations of 30 to 240 minutes, at least 30 minutes between visits, nonnegative costs, a daily meal and mood match, and the overall estimated budget. These checks improve structural consistency; they do not establish real opening hours, transport feasibility, current prices, or venue availability.

Implementation and data management

User interface modules

Home introduces the project and its five mood choices. Planner collects inputs through four steps. Loading shows generation progress and a return-to-choices action. Itinerary presents daily stops, maps, estimated costs, and ratings. Confirm supports printing all days. Shared displays a saved plan through a public share link. About explains the AI flow and design-thinking work.

Backend interfaces

GET /api/health reports backend availability. POST /api/itinerary generates and saves a plan. POST /api/ratings saves a rating, and POST /api/ratings/read reads the creator rating. POST /api/share creates or retrieves a share token. GET /api/shared/:shareToken returns the shared plan without its private feedback code.

Database design

The itineraries table stores a UUID primary key, destination, mood, JSONB plan, feedback-token hash, optional unique share token, and creation timestamp. The ratings table stores one row per itinerary, with itinerary_id as both primary key and foreign key, a rating constrained to 1 through 5, and creation and update timestamps. Deleting an itinerary cascades to its rating.

JSONB allows the complete nested day-and-stop plan to be retained without creating a separate table for every nested item. The one-to-one rating relationship means updating a rating changes the existing feedback record instead of creating repeated votes for the same itinerary.

Access and credential handling

The database schema enables row-level security and revokes direct table access from anonymous and authenticated browser roles. The backend uses its server-side service role. Service keys are kept in local environment settings and Vercel production secrets, excluded from Git, and are not embedded in the browser bundle.

A generated plan has a random private feedback code; only its SHA-256 hash is stored in the database. Rating and share creation requests must pass the matching code. A public share token grants read access to the plan and does not itself grant rating access. The creator code may be retained in browser-tab session storage. Account-based recovery and a share-link revocation interface are not implemented.

Testing and deployment

Checks completed during deployment

Check	Observed result
Production frontend build	Passed with npm run build
Local API routing	Direct health route and Vercel rewrite route passed
Invalid itinerary input	Rejected with HTTP 400
Vercel production deployment	Ready on the published domain
Live backend health	HTTP 200 with status ok
Live itinerary request	Mysore, 1 day, INR 5,000, relaxing returned HTTP 200 and a saved-plan identifier
Static code checking	Existing frontend issues remain; not a clean pass

The live sample request also returned a creator feedback code, consistent with successful generation and persistence. This is a functional smoke check, not proof of recommendation quality, high-load stability, or correctness for every supported destination.

Further verification needed

A full acceptance pass should cover ratings, updating a rating, sharing in a separate browser, PDF export for multiple days, missing or expired links, provider failures, narrow mobile screens, and boundary values. The repository includes service test scripts, but their presence alone does not establish that every scenario passed. Traveler interviews and usability studies remain planned; no user-study results or accuracy percentages are claimed.

Deployment process

The GitHub repository teju7-ai/MoodTrip is connected to the Vercel project mood-trip. Vite builds static assets into dist. The shared Express app is exported from server/app.mjs and invoked through api/index.mjs. Vercel rewrites /api requests to that handler and routes other application paths to index.html for client-side navigation.

Production settings contain OPENROUTER_API_KEY, GEOAPIFY_API_KEY, SUPABASE_URL, and SUPABASE_SECRET_KEY. After saving these settings, the project was redeployed and tested. The public site runs without requiring the developer computer or local server windows to stay open. No credential values belong in this report.

Limitations conclusion and references

Current limitations

- AI estimates are not verified prices. Budgets exclude accommodation, travel, and expenses not listed in the itinerary.
- A fixed 15 km search radius can omit distant attractions and cannot represent an entire state or country.
- Opening hours, transport routes, real travel time, and availability are not checked against live services. Different place IDs may still refer to the same physical venue.
- Free-model availability and response latency vary. A failed validation produces an error instead of a guaranteed itinerary.
- There is no account-based recovery, booking, payment processing, or share-link revocation interface. The current deployment has not been load-tested.

Future improvements

Useful next steps include live routing and opening-hour checks, clearer total-cost coverage, creator accounts and recovery, revocable links, rate limiting, broader automated tests, and structured usability sessions. A later recommendation study could compare mood fit and budget fit across users before considering a learned ranking model.

Conclusion

MoodTrip demonstrates an end-to-end prototype that turns mood and budget inputs into a structured travel outline. Its main technical contribution is the combination of retrieved places, constrained AI output, server validation, persistent plans, and creator-controlled feedback. The deployed application and successful sample request establish basic functionality. Formal evaluation is still needed to determine how useful and reliable its recommendations are for travelers.

Project sources and references

Primary implementation sources: README.md; server/app.mjs; server/places.mjs; server/moodProfiles.mjs; server/generateItinerary.mjs; server/itinerarySchema.mjs; server/feedbackRanking.mjs; server/planStore.mjs; server/ratings.mjs; server/sharing.mjs; supabase/schema.sql; vercel.json; and the React page components in src.

Source repository: https://github.com/teju7-ai/MoodTrip
Live application: https://mood-trip-mu.vercel.app/
Places provider: https://www.geoapify.com/
Map data attribution: https://www.openstreetmap.org/copyright
Report date: 15 October 2026